EFFICIENT 3D PERCEPTION INFERENCE PIPELINE

Po-Lin Chen (polinc) * **ChaoI Tuan (ctuan)** * **Manyung Emma Hon (mehon)** *

ABSTRACT

Deploying 3D perception models to edge devices presents severe latency and energy challenges. This study introduces system-level optimizations—including operator fusion, memory layout transformations, and mixed precision—to the PointPillars pipeline. Through profiling on NVIDIA T4, A10, and H100 GPUs, we demonstrate the hardware scaling effect and prominent performance gain on resource-constrained hardware. Notably, we reduced latency on T4 by 83% and energy cost by 75%, significantly pushing the accuracy-latency-energy frontier.

1 INTRODUCTION

Modern autonomous driving relies heavily on 3D perception models, which face strict latency and energy constraints when deployed on edge devices. While frameworks like OpenPCDet offer modularity, their native PyTorch implementations suffer from heavy framework overhead, frequent Python dispatching, and unoptimized memory access. Furthermore, since not all edge devices possess H100-level compute power, exploring system optimizations to constrained hardware is crucial for scalable deployment. In this research, our main contributions are:

- **In-depth Profiling:** We conduct a comprehensive profiling of the accuracy-latency-energy trade-offs across multiple GPU generations (T4, A10, H100).
- **Hardware Scaling:** We demonstrate that these techniques yield the greatest benefit on constrained hardware, cutting T4 latency by 83% without mAP degradation. This demonstrates the advantage of optimizations for edge devices.
- **Full-System Energy Methodology:** We combine GPU power profiling (NVML) with Intel RAPL CPU measurements to expose system-level energy dynamics (including idle power and CPU processing cost) that GPU-only metrics obscure.
- **Closing to Hardware Limit:** Our pure PyTorch optimizations close 68% of the latency gap to NVIDIA's hand-optimized CUDA-PointPillars without manual CUDA rewrites, demonstrating cross-platform efficiency.

2 PROBLEM

2.1 PointPillars Pipeline Overview

The end-to-end PointPillars pipeline consists of five major stages: Voxelization, Voxel Feature Encoding (VFE), Scatter, 2D Convolutional Backbone, and Detection Head, followed by Post-Processing.

Voxelization. First, the raw LiDAR point cloud is discretized through voxelization (pillarization), where 3D space is partitioned into a regular grid along the XY plane. Each point is assigned to a pillar via coordinate quantization and indexing. This stage is primarily memory-bound, involving integer arithmetic, hashing or array indexing, and irregular memory writes for grouping points.

Voxel feature encoding (VFE). Points within each pillar are processed to extract local geometric features. This stage is characterized by dense floating-point computation combined with reduction operations, where the latter introduces irregular memory access and synchronization overhead.

Scatter. This operator converts sparse pillar features into a dense bird's-eye view (BEV) pseudo-image. Each encoded pillar feature is written to its corresponding 2D grid location. This stage is memory-bound, involving sparse-to-dense mapping, index-based writes, and potential memory access inefficiencies due to non-coalesced patterns.

2D convolutional backbone. This stage is compute-bound and dominated by dense tensor operations such as convolutions, batch normalization, and nonlinear activations. Due to its regular computation pattern, it is highly optimized on modern accelerators.

Detection head. The head of PointPillars is anchor-based, where predefined anchors are tiled across the BEV feature map. For each anchor, the network predicts classification scores and bounding box regression parameters. This stage consists of lightweight convolutional layers followed by dense output generation. While the convolution itself is efficient, the large number of anchors leads to significant output tensor sizes.

Post-processing. Finally, score thresholding and non-maximum suppression (NMS) remove redundant bounding boxes. NMS involves pairwise IoU computations and sorting operations making it memory-bound, especially when the number of candidate boxes is large.

Overall, the pipeline exhibits heterogeneous computational characteristics: voxelization and scatter are memory-bound with irregular access patterns; VFE combines dense computation with reduction operations; the backbone is compute-bound with regular structure; and the detection head and post-processing introduce additional memory and control-flow overhead.

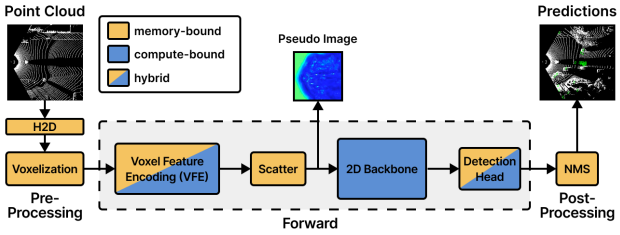

Figure 1. A standard end-to-end PointPillars pipeline

2.2 Problem Statement

We aim to answer three research questions: 1. Can software-level optimization techniques (e.g., operator fusion, memory layout optimizations, custom kernels) bridge the gap between the PyTorch implementation and hardware limits without sacrificing accuracy? 2. How do bottlenecks shift across hardware (T4 vs. A10 vs. H100), and do resource-constraint devices achieve the highest performance gain? 3. What is the accuracy-latency-energy pareto frontier, and how do we evaluate the trade-offs and select from these solutions in practice?

3 Related Work

3.1 Efficient 3D Point Cloud Inference

Recent advancements in 3D point cloud processing have heavily focused on mitigating the severe memory and computational bottlenecks inherent to irregular sparse convolutions. High-performance inference engines, such as TorchSparse (Tang et al., 2022), have demonstrated that replacing naive gather-scatter operations with vectorized, locality-aware memory accesses and adaptive matrix multiplication grouping can significantly boost GPU utilization. Moreover, systematically aligning memory layouts to channels-last (NHWC) and leveraging Automatic Mixed Precision (AMP) enables efficient execution on modern Tensor Cores by satisfying their hardware alignment constraints (NVIDIA, 2020). At the architecture and runtime levels, specialized accelerators like PointAcc (Lin et al., 2021) and sparse convolution frameworks such as SpConv (Yan et al., 2020) explicitly exploit spatial sparsity to reduce off-chip data movement. Furthermore, energy-aware profiling frameworks like Zeus (You et al., 2023) emphasize the critical need to co-optimize latency and power limits to navigate the efficiency–performance trade-off.

3.2 Hardware-Aware Deployment vs. Python-Native Optimization

To bridge the gap between high-level perception frameworks and hardware limits, industry solutions traditionally rely on hardware-specific deployment engines. For example, NVIDIA's CUDA-PointPillars utilizes TensorRT and custom plugins specifically tuned for its accelerators (NVIDIA, 2021b), while Intel provides OpenVINO-optimized PointPillars tailored for its own hardware ecosystem (Intel, 2021). While achieving peak performance, these hardware-bound strategies create rigid translation layers that sacrifice cross-platform flexibility and increase deployment complexity.

3.3 Energy Modeling and Measurement

Accurately quantifying energy consumption is critical for edge deployment. Historically, system energy is analytically modeled as $E_{layer} = E_{comp} + E_{data}$, where memory data movement (E_{data}) heavily dominates the cost (Kandiah et al., 2021; You et al., 2023). However, purely analytical approximations fail to capture dynamic execution behaviors, such as idle power drainage. Therefore, our work shifts from theoretical FLOP-based models to real-time hardware power profiling to capture ground-truth energy footprints.

4 Method

4.1 Profiling & Energy Modeling

To accurately quantify the performance of our optimized PointPillars pipeline, we established a rigorous multi-dimensional profiling framework. We adopted the core OpenPCDet framework, while simulating a realistic autonomous driving edge-deployment scenario by streaming point cloud frames at a fixed 10Hz frequency with a batch size of 1.

Latency profiling. We utilize NVIDIA Nsight Systems to capture fine-grained execution timelines. To thoroughly understand system bottlenecks and execution consistency, we extract the median, p95, and p99 latency distributions. The end-to-end latency is explicitly decomposed into 6 sequential functional stages: Voxelization, Voxel Feature Encoding (VFE), Scatter, 2D BEV Backbone, Anchor Head, and Post-Processing.

Energy measurement & fairness. To capture the dynamic energy footprint of each pipeline stage, we utilize the NVIDIA Management Library (NVML) (NVIDIA, 2021a) to perform high-resolution, real-time polling of GPU power sensors during inference on Modal A10G. To reveal full system-level effects that NVML cannot capture, we supplement this with Intel RAPL measurements on a local RTX 3080 Ti edge GPU (43 W TDP) to profile CPU package energy alongside the GPU. For preprocessing fairness on Modal (where RAPL is unavailable), CPU cost is estimated at $\approx$10 W $\times$ preprocessing time. Since this is $<$1% of the A10G's GPU frame energy, the GPU-only ranking remains valid.

4.2 Operator Fusion and Overhead Reduction

To reduce framework-induced inefficiencies, we integrate `torch.compile` to transform eager execution into an optimized computation graph. This enables operator fusion, reducing the number of kernel launches and minimizing intermediate tensor materialization. In the original pipeline, operators are executed independently, leading to excessive Python dispatch overhead, frequent kernel launches, and suboptimal GPU utilization. These issues are particularly severe in stages such as Scatter, where many small operations are executed sequentially.

With graph-level compilation, adjacent operators are fused into larger kernels, effectively amortizing kernel launch overhead and improving execution efficiency. Additionally, fusion reduces redundant memory traffic by eliminating unnecessary reads and writes of intermediate tensors. To quantitatively analyze these effects, we instrument the execution using NVIDIA Nsight Systems, capturing kernel launch counts, GPU utilization, and execution timelines across pipeline stages.

Table 1. CUDA Launches and MemOps

METRIC	BASELINE	COMPILED	EXPLANATION
cudaLaunchKernel	16,752	10,014	-40% (fusion)
cuLaunchKernel	2	1,583	driver API (JIT)
cudaMemcpyAsync	1,976	1,952	-
cuModuleLoadData	0	29	JIT modules
H2D (MB)	272	272	-
D2D (MB)	4,472	632	-85% (fusion)

4.3 Memory Layout Optimization

HWC-style scatter. Scatter is dominated by irregular index-based writes. We optionally materialize the grid buffer in a spatial-major layout so each occupied cell's channel vector is written contiguously.

Channels-last BEV convolutions. The 2D BEV backbone is a stack of `Conv2d` / `BatchNorm2d` / ReLU blocks. We optionally convert the BEV feature map to PyTorch's channels-last (NHWC) memory format so that cuDNN-style kernels can exploit layout that matches modern GPU Tensor Core constraints.

4.4 Preprocessing Offloading

In the default OpenPCDet pipeline, voxelization is implemented in the data loader and executed on the CPU. This design incurs significant preprocessing latency and redundant host-to-device (H2D) data transfers, as both raw points and intermediate voxelized representations must be transferred to the GPU.

Following the design of CUDA-PointPillars, we offload voxelization to the GPU by implementing a CUDA-based preprocessing pipeline. This optimization provides two key benefits: reduced latency and minimized data movement.

Reduced latency. Offloading voxelization to the GPU significantly reduces preprocessing overhead. Compared to the baseline CPU implementation, preprocessing latency is reduced by approximately 3 $\times$.

Minimized data movement. Originally both raw points ($\approx$ 1.6MB) and voxelized intermediate representations ($\approx$ 20MB) are transferred to the GPU. After offloading, only raw point clouds are transferred, reducing memory traffic.

4.5 TensorRT Inference Engine

NVIDIA's CUDA-PointPillars **(M5)** serves as the hardware-specific baseline, replacing the OpenPCDet PyTorch stack with a fully hand-written C++ CUDA pipeline. All five stages—voxelization, Voxel Feature Encoding (VFE), scatter, 2D backbone, and post-processing NMS—are implemented as dedicated CUDA kernels, eliminating Python dispatching overhead, JIT recompilation, and PyTorch tensor-management costs entirely.

The neural network portion (VFE through Anchor Head) is compiled offline into a static TensorRT engine. TensorRT applies layer fusion (combining adjacent Conv2d, BatchNorm, and ReLU operations into single kernels), selects hardware-tuned cuDNN kernel implementations, and preallocates workspace buffers, removing all runtime kernel-selection overhead present in the PyTorch path. For FP16 inference, the engine is compiled with full Tensor Core alignment, providing native FP16 execution rather than the dynamic AMP wrapping used in M0–M4. The custom voxelization kernels (`generateVoxels`, `generateFeatures`) and CUDA-based post-processing (`doPostprocessCuda`) further reduce host-device synchronization barriers that fragment GPU utilization in the Python pipeline.

Together, these optimizations yield a mean A10G latency of

6.00 ms (FP32) and 3.34 ms (FP16), compared to 10.37 ms for **M4**—a 3.1× speedup at FP16—while delivering the highest energy efficiency across all evaluated configurations at 480 mJ per sample.

4.6 Mixed Precision

We use PyTorch AMP to execute most compute-intensive operations in FP16 while retaining FP32 for numerically sensitive parts. This typically reduces memory traffic and latency via Tensor Core acceleration.

5 Evaluation

5.1 Experimental Setup

As summarized in Table 2, methods M0 through M4 represent our progressive optimizations, while M5 serves as the industry-standard hardware-specific baseline. Performance evaluations were conducted on NVIDIA T4, A10, and H100 GPUs on the Modal platform, while accuracy assessed using the KITTI dataset.

Table 2. Methods Evaluated in this Study

Method	Description	Section
M0 (Baseline)	OpenPCDet implementation.	-
M1 (+ Compiled)	Graph-level fusion and overhead reduction with `torch.compile`.	§4.2
M2 (+ NHWC)	Channel-last memory layout for `Scatter` and `Conv2D`.	§4.3
M3 (+ Offload)	Voxelization offloaded from host CPU to GPU.	§4.4
M4 (All Combined)	Integration of M1–M3.	-
M5 (TensorRT)	Hardware-specific CUDA-PointPillars implementation.	§4.5

5.2 Accuracy

As shown in Table 3, our progressive optimizations (M1 through M5)—including memory layout changes and mixed precision (AMP)—successfully maintained the baseline KITTI Car@R11 mAP without any meaningful degradation.

Table 3. Model performance on the KITTI Dataset.

Precision	Car@R11 mAP	Car@R40 mAP
FP32	84.81	81.46
AMP	84.79	81.45

Table 4. Mean, P95, and P99 latency on A10 GPU (ms).

	Mean			P95			P99		
	FP32	AMP	FP16	FP32	AMP	FP16	FP32	AMP	FP16
M0	15.71	13.85	-	16.35	14.73	-	17.65	16.30	-
M1	15.37	12.65	-	15.82	13.92	-	16.76	15.74	-
M2	17.88	12.95	-	18.36	13.71	-	19.59	14.82	-
M3	13.16	10.51	-	13.65	11.13	-	14.05	11.50	-
M4	12.31	10.37	-	12.82	11.60	-	13.33	12.68	-
M5	6.00	-	**3.34**	7.01	-	**4.38**	7.82	-	**5.15**

5.3 Latency

Latency analysis highlights the cumulative impact of our optimizations on framework and memory bottlenecks, with detailed mean, p95, and p99 distributions summarized in Table 4. Starting from the M0 baseline's mean latency of 15.71 ms, the transition to Mixed Precision (AMP) reduces execution time to 13.85 ms. As shown in Figure 2, the most significant gain occurs in M3, where offloading voxelization to the GPU drops mean latency to 10.51 ms. This shift eliminates the CPU-bound preprocessing bottleneck and minimizes redundant data movement.

Memory layout (M2) reflects the trade-off between layout conversion overhead and hardware acceleration. In FP32, latency slightly increases to from 15.71 ms to 17.88 ms because the transposition cost exceeds cuDNN speedups on PointPillar's relatively small BEV feature maps. Conversely, in AMP, the NHWC format is required to satisfy **Tensor Core** alignment constraints. The massive computational gains from Tensor Core acceleration effectively let M2 outperform the baseline at 12.95 ms.

Ultimately, the integrated pipeline (M4) achieves a 10.37 ms latency, significantly closing the performance gap toward the 3.34 ms hardware-specific TensorRT (M5) ceiling.

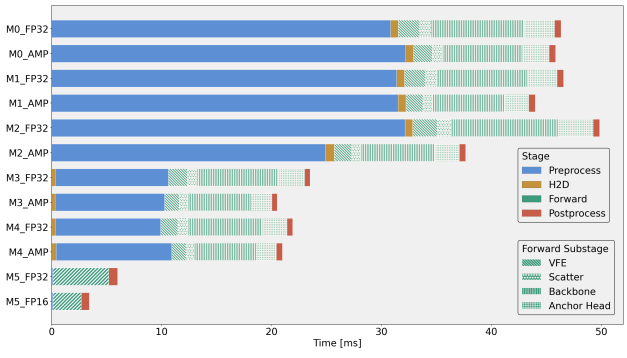

Figure 2. Latency breakdown across pipeline stages on A10 GPU.

5.4 Energy

To rigorously evaluate our optimizations, we progress from standard GPU-only power profiling to a holistic full-system energy deep dive on edge hardware.

GPU-Only Metrics and Measurement Bias. We first evaluate GPU energy efficiency on the Modal platform using NVML. As shown in Figure 3, **M1_AMP** effectively improves energy efficiency (Samples/J) by 47% through the compounded effects of kernel fusion (reducing D2D traffic by 85%) and reduced per-op power from mixed precision using Tensor Cores. **M5_FP16** is the most energy-efficient method which consumes only 480 mJ per sample.

However, analyzing **M3** and **M4** reveals a fundamental measurement bias in GPU-only metrics. By offloading voxelization from the host to the device, the GPU assumes additional workload. This artificially inflates the GPU's energy footprint, making M3 appear to yield only a marginal +7% gain, while M4 appears significantly less efficient (-28%) due to JIT recompilation overhead. In cloud environments where CPU power is often obscured, these metrics inherently penalize system-level offloading optimizations.

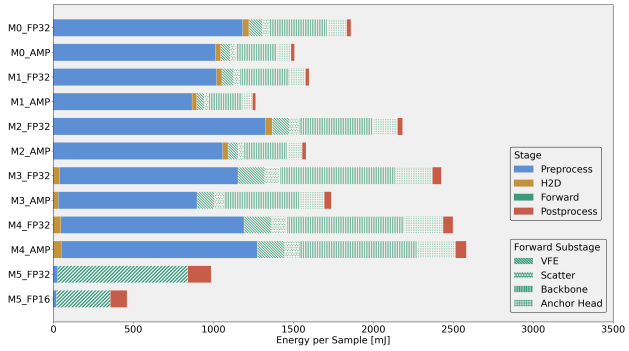

Figure 3. GPU-only energy efficiency on A10 GPU.

Full-System Energy Dynamics on Edge Hardware. Unlike the GPU-only analysis, we measure full-system energy (CPU package + platform) over a complete 10Hz inference cycle, explicitly accounting for idle periods between frames. We use Intel RAPL to capture CPU and platform (PSYS) power, enabling a deployment-realistic energy breakdown.

Figure 4 shows that total energy is dominated by platform overhead, followed by CPU idle power and GPU idle power. Comparing **M0**, **M1**, **M3**, and **M4**, we observe a consistent trend: as latency decreases, active execution time shrinks while idle time increases within the fixed 100 ms cycle.

This shift is beneficial because active power is significantly higher than idle power. The CPU draws ∼78 W when active but only ∼11.6 W when idle, while the GPU consumes 19.4 W active vs. 18.2 W idle. As a result, reducing active time directly lowers total system energy.

Notably, **M3** and **M4** offload preprocessing to the GPU. Although this increases GPU workload, it replaces expensive CPU computation (∼78 W) with significantly lower-power GPU execution (∼19.4 W). This trade-off leads to a net reduction in total system energy.

Overall, these results confirm that in edge deployments, energy efficiency is governed by reducing system-wide active time rather than minimizing per-device compute alone.

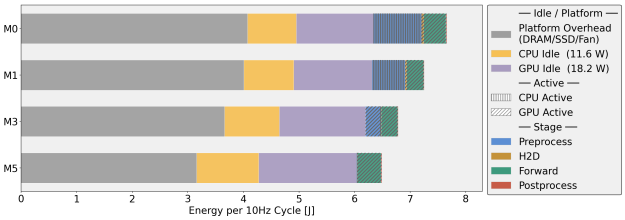

Figure 4. Full-system energy breakdown on a RTX 3080 Ti Laptop GPU (configured at ∼43 W).

5.5 Hardware Scaling Analysis

Figure 5 shows the accuracy-latency-energy Pareto frontier across T4, A10G, and H100. Since mAP remains consistent across all configurations, we focus on latency and energy.

The full optimization stack **M5_FP16** delivers substantial gains on every GPU generation. On the T4, full-frame latency drops by 83% and system energy falls by 75%, enabling real-time 10 Hz inference with significant headroom. On A10G and H100, latency reduces by 72% and 83% respectively, confirming that the combined effect of TensorRT compilation, custom CUDA kernels, and native FP16 execution scales across hardware generations.

Critically, the absolute latency savings are largest on the T4, which matters most for real-world deployment: autonomous driving edge devices operate on T4-class hardware, not flagship data center GPUs. Software-level optimization therefore has its highest practical impact exactly where constrained hardware makes it most necessary.

5.6 Roofline Analysis

Roofline analysis confirms that the PointPillars forward pass is fundamentally DRAM-bandwidth limited across all GPU generations. As shown in Figure 6, each variant's arithmetic intensity (FLOP per byte, measured via NCU) is plotted against its achieved GFLOP/s, with the memory-bandwidth roof and peak compute ceiling as reference. The ridge point separates memory-bound (left) from compute-bound (right) regimes.

All M0–M4 variants are memory-bound. Every variant

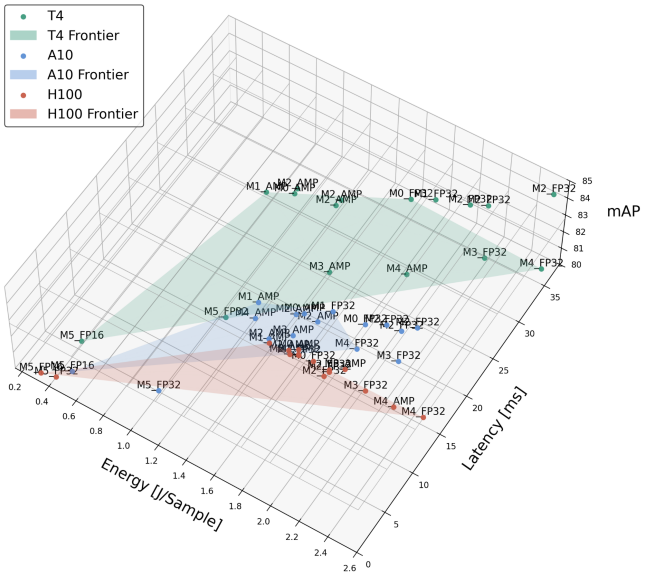

Figure 5. Accuracy-latency-energy pareto frontier across GPUs.

falls to the left of the ridge point, with arithmetic intensity ranging from 6 to 21 FLOP/B. On the T4, the ridge point is $\approx$ 27 FLOP/B (8.1 TFLOP/s $\div$ 300 GB/s), and no variant exceeds it. The forward pass is therefore bottlenecked by DRAM bandwidth, not compute capacity; reducing memory traffic is the primary lever for further improvement.

AMP raises arithmetic intensity $\sim$1.7$\times$. FP16 activations are half the size of FP32, so for the same computation fewer bytes are transferred, directly increasing intensity. On the T4, **M0** rises from 7.9 to 13.9 FLOP/B and **M1** from 9.0 to 20.6 FLOP/B under AMP. This hardware-level shift provides the mechanistic explanation for the latency and energy gains reported in Sections 5.3 and 5.4.

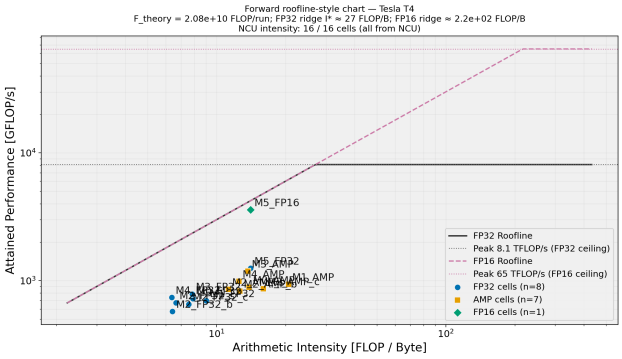

Figure 6. Roofline analysis on T4 GPU

6 Conclusion

In this work, we systematically optimized the PointPillars 3D perception pipeline to address the latency and energy constraints of edge deployment. By integrating techniques such as operator fusion, NHWC memory layout, voxelization offloading, and mixed precision, we achieved substantial reductions in latency and energy consumption while maintaining mAP accuracy. Crucially, our cross-hardware validation across NVIDIA T4, A10, and H100 GPUs demonstrated that these optimizations **yield the most dramatic performance gains on edge hardware**—reducing T4 latency by 83% and energy cost by 75%—proving their immense value for scalable autonomous driving deployment.

References

Intel. Deploying pointpillars with openvino toolkit. https://docs.openvino.ai/latest/openvino_docs_model_zoo.html, 2021.

Kandiah, V., Peverelle, S., Khairy, M., Pan, J., Manjunath, A., Rogers, T. G., Aamodt, T. M., and Hardavellas, N. AccelWattch: A Power Modeling Framework for Modern GPUs. In *54th Annual IEEE/ACM International Symposium on Microarchitecture (MICRO '21)*, pp. 1–16, 2021.

Lin, Y., Zhang, Z., Tang, H., Wang, H., and Han, S. PointAcc: Efficient Point Cloud Accelerator. In *54th Annual IEEE/ACM International Symposium on Microarchitecture (MICRO '21)*, 2021.

NVIDIA. Programming tensor cores in cuda 11. https://developer.nvidia.com/blog/programming-tensor-cores-cuda-11/, 2020.

NVIDIA. NVIDIA Management Library (NVML) API Reference Guide. https://developer.nvidia.com/nvidia-management-library-nvml, 2021a.

NVIDIA. Nvidia tensorrt. https://developer.nvidia.com/tensorrt, 2021b.

Tang, H., Liu, Z., Li, X., Lin, Y., and Han, S. Torchsparse: Efficient point cloud inference engine. In *Proceedings of Machine Learning and Systems (MLSys)*, 2022.

Yan, Y., Mao, Y., and Li, B. Spconv: Spatially sparse convolution library. In *CVPR Workshop*, 2020.

You, J., Chung, J.-W., and Chowdhury, M. Zeus: GPU Energy as a First-Class Resource in DNN Training. In *20th USENIX Symposium on Networked Systems Design and Implementation (NSDI '23)*, 2023.